

COMPSCI 701:
Creating Maintainable Software
Lecture 5.1: Functions and Maintainability I

Yu-Cheng Tu, Ewan Tempero
School of Computer Science

Agenda

- Agenda

- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- Do one thing
- Self Documenting
- Abstraction Level
- Key Points

- Admin
 - Updated slides for Lecture 4.2
 - Assignment 2
 - 19 “tests” — all 4 variants for each test
- Class Exercise
- Functions and Maintainability

Class Exercise

- Agenda
- **Class Exercise**
- Functions
- Structured Programming
- Martin
- Refactoring
- Do one thing
- Self Documenting
- Abstraction Level
- Key Points

- Compare NoughtsAndCrosses-SFV with NoughtsAndCrosses-MFV (from Modules on Canvas)
 1. *Quickly* decide which is the more **comprehensible**
 2. The effort required to perform a comprehension task depends on the tasks
 - Is there are task for which it would be easier for **SFV** compared with MFV?
 - Is there are task for which it would be easier for **MFV** compared with SFV?

Explain your choices in terms of the comprehension model.

Maintainable Functions (and Methods)

- Agenda
- Class Exercise
- **Functions**
- Structured Programming
- Martin
- Refactoring
- Do one thing
- Self Documenting
- Abstraction Level
- Key Points

- The idea of breaking things into separately-callable units has been around a long time—probably the 1940s
- The original goal was almost certainly to reduce code duplication and memory use
- The idea of doing so to make the code **easier to maintain** is more recent (1960s or earlier)—“structured programming”

Structured Programming

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- Do one thing
- Self Documenting
- Abstraction Level
- Key Points

- essentially disciplined use of “go to” statements

```
10 READ A FROM KEYBOARD
20 READ B FROM KEYBOARD
30 IF A > B GOTO 200
40 IF A = B GOTO 100
50 PRINT "Less than"
60 GOTO 150
100 PRINT "Equals"
110 GOTO 1000
150 GOTO 1000
200 PRINT "Greater than"
1000 END PROGRAM
```

- Edsger W. Dijkstra. 1968. Letters to the editor: go to statement considered harmful. *Communications of the ACM* 11, 3 (March 1968), 147–148. DOI:<https://doi.org/10.1145/362929.362947>—advocated removing “go to” from languages, essentially from a comprehensibility perspective
- languages today, if they have “go to” statements, are only in a very restricted form (e.g. condition, loop statements)
- Structured Programming imposes more discipline on how decisions are made about code is organised
- One form of it forbids more than one return from any function

Martin on Functions

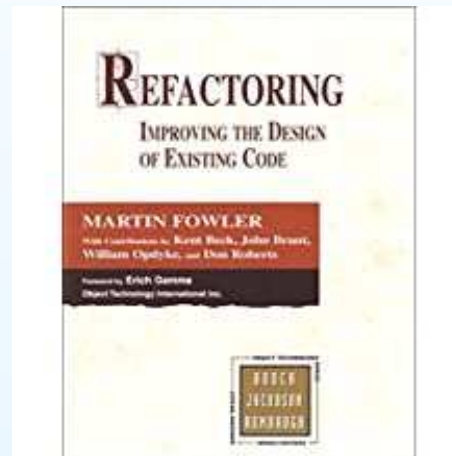
- Agenda
- Class Exercise
- Functions
- Structured Programming
- **Martin**
- Refactoring
- Do one thing
- Self Documenting
- Abstraction Level
- Key Points

- Small
- Do One Thing (“cohesion”)
- One Level of Abstraction per Function
- Reading Code from Top to Bottom
- Use Descriptive Names
- Function Arguments
- Have No Side Effects

How to get good functions?

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- **Refactoring**
- Do one thing
- Self Documenting
- Abstraction Level
- Key Points

- (1) Avoid problems in the first place
- (2) If got problems, **refactor**
 - ⇒ change code to improve code quality without changing functionality
 - Martin Fowler (feat. Kent Beck) *Refactoring: Improving the Design of Existing Code*. Addison-Wesley (2012)



- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- **Do one thing**
- Self Documenting
- Abstraction Level
- Key Points

Do one thing

```
// original 75(!) line function
public static void play(IO io) {
    Board board = new Board(io);
    board.initialiseBoard();
    board.displayBoard();
    int rowChoice = 0;
    int colChoice = 0;
    String reason = " (quit)";
    turn: while (rowChoice >= 0) {
        rowChoice = io.readInteger("Player X choose row: ", 0, 2, -1, "q");
        if (rowChoice < 0) {
            break;
        }
        colChoice = io.readInteger("Player X choose col: ", 0, 2, -1, "q");
        if (colChoice < 0) {
            break;
        }
        boolean result = board.play(rowChoice,colChoice,'X');
        while (!result) {
            rowChoice = io.readInteger("Player X choose row: ", 0, 2, -1, "q");
            if (rowChoice < 0) {
                break turn;
            }
            colChoice = io.readInteger("Player X choose col: ", 0, 2, -1, "q");
            if (colChoice < 0) {
                break turn;
            }
            result = board.play(rowChoice,colChoice,'X');
        }
        .....
    }
}
```

- Doing one thing?
 - “Playing the game” (one thing)
 - “Initialising the board, displaying the board, getting player X move, ...” (many things)
 - **Useful heuristic — identify duplication**

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- **Do one thing**
- Self Documenting
- Abstraction Level
- Key Points

Do one thing

```
public static void play(IO io) {
    Board board = new Board(io);
    ....
    String reason = " (quit)";
turn: while (rowChoice >= 0) {
    rowChoice = io.readInteger("Player X choose row: ", 0, 2, -1, "q");
    if (rowChoice < 0) {
        break;
    }
    ....
    if (board.checkGameWon()) {
        io.println("Player X wins!");
        reason = " (X wins)";
        break;
    }
    if (board.checkDraw()) {
        io.println("Draw!");
        reason = " (draw)";
        break;
    }
    ....
    if (board.checkGameWon()) {
        io.println("Player O wins!");
        reason = " (O wins)";
        break;
    }
    if (board.checkDraw()) {
        io.println("Draw!");
        reason = " (draw)";
        break;
    }
}
io.println("Game over" + reason);
}
```

Do one thing

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- **Do one thing**
- Self Documenting
- Abstraction Level
- Key Points

```
private static String gameOverWithDraw(IO io, Board board) {  
    if (board.checkDraw()) {  
        io.println("Draw!");  
        io.println("Game over (draw)");  
        return " (draw)";  
    }  
    return ""; ??????  
}
```

- “small”
- “doing one thing” — dealing with when game ends in a draw
- But, just moving duplicate code into a function doesn’t always work
- Some thought has to be given about how information gets in, and out, of the function

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- **Do one thing**
- Self Documenting
- Abstraction Level
- Key Points

Do one thing

```
public static void play(IO io) {
    Board board = new Board(io);
    .....
    String reason = " (quit)";
turn: while (rowChoice >= 0) {
    rowChoice = io.readInteger("Player X choose row: ", 0, 2, -1, "q");
    if (rowChoice < 0) {
        io.println("Game over" + reason);
        break;
    }
    ....
    if (board.checkGameWon()) {
        io.println("Player X wins!");
        reason = " (X wins)";
        io.println("Game over" + reason);
        break;
    }
    if (board.checkDraw()) {
        io.println("Draw!");
        reason = " (draw)";
        io.println("Game over" + reason);
        break;
    }
    ....
    if (board.checkGameWon()) {
        io.println("Player O wins!");
        reason = " (O wins)";
        io.println("Game over" + reason);
        break;
    }
    if (board.checkDraw()) {
        io.println("Draw!");
        reason = " (draw)";
        io.println("Game over" + reason);
        break;
    }
    }
}
```

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- **Do one thing**
- Self Documenting
- Abstraction Level
- Key Points

Do one thing

```

public static void play(IO io) {
    Board board = new Board(io);
    .....
    turn: while (rowChoice >= 0) {
        rowChoice = io.readInteger("Player X choose row: ", 0, 2, -1, "q");
        if (rowChoice < 0) {
            io.println("Game over" + " (quit)");
            break;
        }
        ....
        if (board.checkGameWon()) {
            io.println("Player X wins!");
            io.println("Game over" + " (X wins)");
            break;
        }
        if (board.checkDraw()) {
            io.println("Draw!");
            io.println("Game over" + " (draw)");
            break;
        }
        ....
        if (board.checkGameWon()) {
            io.println("Player O wins!");
            io.println("Game over" + " (O wins)");
            break;
        }
        if (board.checkDraw()) {
            io.println("Draw!");
            io.println("Game over" + " (draw)");
            break;
        }
    }
}

```

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- **Do one thing**
- Self Documenting
- Abstraction Level
- Key Points

Do one thing

```
private static boolean gameOverWithDraw(IO io, Board board) {  
    if (board.checkDraw()) {  
        io.println("Draw!");  
        io.println("Game over (draw)");  
        return true;  
    }  
    return false;  
}
```

- Number of parameters: 2
- Number of returns: 2
- Often refactoring is a case of one step backwards, two steps forwards!

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- **Do one thing**
- Self Documenting
- Abstraction Level
- Key Points

Do one thing

```
public static void play(IO io) {
    Board board = new Board(io);
    .....
    while (rowChoice >= 0) {
        rowChoice = io.readInteger("Player X choose row: ", 0, 2, -1, "q");
        if (rowChoice < 0) {
            io.println("Game over" + " (quit)");
            break;
        }
        ....
        if (board.checkGameWon()) {
            io.println("Player X wins!");
            io.println("Game over" + " (X wins)");
            break;
        }
        if (gameOverWithDraw(io, board)) {
            break;
        }
        if (board.checkDraw()) {
            io.println("Draw!");
            io.println("Game over" + " (draw)");
            break;
        }
        ....
        if (board.checkGameWon()) {
            io.println("Player O wins!");
            io.println("Game over" + " (O wins)");
            break;
        }
        if (gameOverWithDraw(io, board)) {
            break;
        }
        if (board.checkDraw()) {
            io.println("Draw!");
            io.println("Game over" + " (draw)");
            break;
        }
    }
}
```

Do one thing

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- **Do one thing**
- Self Documenting
- Abstraction Level
- Key Points

```
private static boolean gameOverWithWinner(IO io, Board board, String player) {
    if (board.checkGameWon()) {
        io.println("Player " + player + " wins!");
        io.println("Game over (" + player + " wins)");
        return true;
    }
    return false;
}

private static int getPlayerChoice(IO io, String player, String coordinate) {
    int choice = io.readInteger("Player " + player + " choose " + coordinate + ": ", 0, 2, -1, "q");
    if (choice < 0) {
        io.println("Game over (quit)");
    }
    return choice;
}
```

- small
- Does one thing
- Number of parameters: 3
- Number of returns: 2

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- Do one thing
- Self Documenting
- Abstraction Level
- Key Points

Do one thing

```
public static void play(IO io) {
    Board board = new Board(io);
    ....
    while (true) {
        rowChoice = getPlayerChoice(io, "X", "row");
        if (rowChoice < 0) {
            return;
        }
        colChoice = getPlayerChoice(io, "X", "col");
        if (colChoice < 0) {
            return;
        }
        boolean result = board.play(rowChoice,colChoice,'X');
        while (!result) {
            rowChoice = getPlayerChoice(io, "X", "row");
            if (rowChoice < 0) {
                return;
            }
            colChoice = getPlayerChoice(io, "X", "col");
            if (colChoice < 0) {
                return;
            }
        }
        result = board.play(rowChoice,colChoice,'X');
    }
    ....
    rowChoice = getPlayerChoice(io, "O", "row");
    if (rowChoice < 0) {
        return;
    }
    colChoice = getPlayerChoice(io, "O", "col");
    if (colChoice < 0) {
        return;
    }
    result = board.play(rowChoice,colChoice,'O');
    while (!result) {
        rowChoice = getPlayerChoice(io, "O", "row");
        if (rowChoice < 0) {
            return;
        }
        ....
    }
}
```

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- **Do one thing**
- Self Documenting
- Abstraction Level
- Key Points

Do one thing

```
private static boolean playerMove(IO io, Board board, char player) {
    int rowChoice = getPlayerChoice(io, player, "row");
    if (rowChoice < 0) {
        return true;
    }
    int colChoice = getPlayerChoice(io, player, "col");
    if (colChoice < 0) {
        return true;
    }
    boolean result = board.play(rowChoice,colChoice,player);
    while (!result) {
        rowChoice = getPlayerChoice(io, player, "row");
        if (rowChoice < 0) {
            return true;
        }
        colChoice = getPlayerChoice(io, player, "col");
        if (colChoice < 0) {
            return true;
        }
        result = board.play(rowChoice,colChoice,player);
    }
    board.displayBoard();
    if (gameOverWithWinner(io, board, player)) {
        return true;
    }
    if (gameOverWithDraw(io, board)) {
        return true;
    }

    return false;
}
```

- Number of returns: 7
- Not that small
- Doing one thing?

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- **Do one thing**
- Self Documenting
- Abstraction Level
- Key Points

Reducing Returns

Before

```
private static boolean gameOverWithDraw(IO io, Board board) {  
    if (board.checkDraw()) {  
        io.println("Draw!");  
        io.println("Game over (draw)");  
        return true;  
    }  
    return false;  
}
```

After

```
private static boolean gameOverWithDraw(IO io, Board board) {  
    boolean gameIsOver = false;  
    if (board.checkDraw()) {  
        io.println("Draw!");  
        io.println("Game over (draw)");  
        gameIsOver = true;  
    }  
    return gameIsOver;  
}
```

- Not obviously worse in comprehensibility
- But not easy to apply in other cases (e.g. `playerMove()`)
- Does the benefit from having only one return outweigh the contortions needed to achieve it?
- What does the program comprehension model say about “only one return”?

“Explain Yourself in Code”

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- Do one thing
- **Self Documenting**
- Abstraction Level
- Key Points

```
private static boolean playerMove(IO io, Grid grid, char player) {
    boolean legalMove = false;
    while (!legalMove) {
        int[] choice = getPlayerChoice(io, player);
        if ((choice[0] < 0 || choice[1] < 0)) {
            return true;
        }
        legalMove = grid.play(choice[0], choice[1], player);
    }
    grid.display();
    if (gameOverWithWinner(io, grid, player)) {
        return true;
    }
    if (gameOverWithDraw()) {
        return true;
    }
    return false;
}
```

- Conditions are a common source of confusion

“Explain Yourself in Code”

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- Do one thing
- Self Documenting
- Abstraction Level
- Key Points

```
private static boolean choseQuit(int[] choice)
    return (choice[0] < 0 || choice[1] < 0);
```

```
private static boolean playerMove(IO io, Grid grid, char player) {
    boolean legalMove = false;
    while (!legalMove) {
        int[] choice = getPlayerChoice(io, player);
        if (choseQuit(choice)) {
            return true;
        }
        legalMove = grid.play(choice[0], choice[1], player);
    }
    grid.display();
    if (gameOverWithWinner(io, grid, player)) {
        return true;
    }
    if (gameOverWithDraw()) {
        return true;
    }
    return false;
}
```

- Use small functions to create “self-documenting” code

“Explain Yourself in Code”

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- Do one thing
- Self Documenting
- Abstraction Level
- Key Points

```
private static boolean choseQuit(int[] choice)
    return (choice[0] < 0 || choice[1] < 0);
```

```
private static boolean playerMove(IO io, Grid grid, char player) {
    boolean legalMove = false;
    while (!legalMove) {
        int[] choice = getPlayerChoice(io, player);
        if (choseQuit(choice)) {
            return true;
        }
        legalMove = grid.play(choice[0], choice[1], player);
    }
    grid.display();
    if (gameOverWithWinner(io, grid, player)) {
        return true;
    }
    if (gameOverWithDraw()) {
        return true;
    }
    return false;
}
```

- Use small functions to create “self-documenting” code
- What does it mean when `playerMove()` returns true

“Explain Yourself in Code”

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- Do one thing
- Self Documenting
- Abstraction Level
- Key Points

```
private static final boolean GAME_FINISHED = true
private static boolean playerMove(IO io, Grid grid, char player) {
    boolean legalMove = false;
    while (!legalMove) {
        int[] choice = getPlayerChoice(io, player);
        if (choseQuit(choice)) {
            return GAME_FINISHED;
        }
        legalMove = grid.play(choice[0], choice[1], player);
    }
    grid.display();
    if (gameOverWithWinner(io, grid, player)) {
        return GAME_FINISHED;
    }
    if (gameOverWithDraw()) {
        return GAME_FINISHED;
    }
    return ! GAME_FINISHED;
}
```

- Avoid use of literal values (not about functions but worth remembering)

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- Do one thing
- Self Documenting
- Abstraction Level
- Key Points

One Level of Abstraction per Function

- (Martin) Avoid having “high level” concepts and “low level” concepts in the same function
 - E.g. Code that expresses what is happening in terms of domain concepts (e.g. the changing a cell in the grid) versus code that implements a concept (e.g. details about the values in the array mean)

Multiple levels means function is not “doing one thing”

- Hard to reliably identify levels of abstraction
- An approach is to use levels of indentation as an heuristic — the more indented, the lower the level of abstraction

Reducing number of levels of abstraction

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- Do one thing
- Self Documenting
- Abstraction Level
- Key Points

```
private static boolean playerMove(IO io, Grid grid, char player) {  
    boolean legalMove = false;  
    while (!legalMove) {  
        int[] choice = getPlayerChoice(io, player);  
        if (choseQuit(choice)) {  
            return true;  
        }  
        legalMove = grid.play(choice[0], choice[1], player);  
    }  
    grid.display();  
    if (gameOverWithWinner(io, grid, player)) {  
        return true;  
    }  
    if (gameOverWithDraw()) {  
        return true;  
    }  
    return false;  
}
```

- Look for deeply nested code

Reducing number of levels of abstraction

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- Do one thing
- Self Documenting
- Abstraction Level
- Key Points

```
private static boolean playerMove(IO io, Grid grid, char player) {
    if (playLegalMove(io, grid, player)) {
        return GAME_FINISHED;
    }
    grid.display();
    if (gameOverWithWinner(io, grid, player)) {
        return GAME_FINISHED;
    }
    if (gameOverWithDraw()) {
        return GAME_FINISHED;
    }
    return ! GAME_FINISHED;
}

private static boolean playLegalMove(IO io, Grid grid, char player) {
    boolean legalMove = false;
    while (!legalMove) {
        int[] choice = getPlayerChoice(io, player);
        if (choseQuit(choice)) {
            return GAME_FINISHED;
        }
        legalMove = grid.play(choice[0], choice[1], player);
    }
    return ! GAME_FINISHED;
}
```

- How has comprehensibility improved?
- Less code to read (for some tasks) $\Rightarrow$ assimilation process takes less time
- Block of coherent code now mapped to single name (`playLegalMove()`) — if name is “meaningful” then mental model access more efficient
- How far should this go? — `playLegalMove()` has same levels of nesting as the original `playerMove()`!

- Agenda
- Class Exercise
- Functions
- Structured Programming
- Martin
- Refactoring
- Do one thing
- Self Documenting
- Abstraction Level
- Key Points

Key Points

- Use of many small **good** functions can aid comprehension (at least)
 - because their (meaningful!) names indicate the **one thing** that they do (self documenting code!)
 - often the functions with the lower levels of abstraction do not need to be considered, reducing the need to go to the external representation (implementation of those functions)
- Refactor to improve use of functions
- Refactor in multiple steps